

XII. CSS Grid — mise en page 2D

GRID EN UNE PHRASE

Un système de mise en page CSS pour créer des grilles **EN DEUX DIMENSIONS** (lignes ET colonnes). Idéal pour dashboards, galeries, pages entières.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Les bases : display: grid**
- **B. Unités fr et fonction repeat()**
 - 1. L'unité fr (fraction)
 - 2. La fonction repeat()
- **C. Galerie responsive magique**
 - auto-fit vs auto-fill
- **D. Placement des éléments**
 - 1. grid-column et grid-row
 - 2. span — alternative plus lisible
 - 3. justify-items et align-items en Grid
- **E. grid-template-areas — layout nommé**
- **F. Flexbox ou Grid — comment choisir ?**
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Construire** une grille avec `grid-template-columns`, `grid-template-rows`, `gap`.
- **Utiliser** l'unité `fr` et la fonction `repeat()` pour des grilles propres.
- **Créer** une galerie responsive **sans media query** grâce à `auto-fit` + `minmax`.
- **Placer** précisément un élément avec `grid-column` / `grid-row` ou `span`.
- **Décrire** un layout complexe avec `grid-template-areas`.
- **Choisir** Grid (2D) ou Flexbox (1D) selon le besoin.

2. En pratique — situation professionnelle

Le designer vous livre une maquette de dashboard : header, sidebar, contenu principal, footer — le tout en 2 dimensions. Grid règle ce layout en 10 lignes de CSS lisibles, là où Flexbox nécessiterait des imbrications artificielles.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Layout 2D (dashboard, page entière, galerie)	Aligner une ligne d'éléments → préférer Flexbox (chap. XI)
Galerie responsive <code>auto-fit</code> + <code>minmax</code> (sans media query)	Compatibilité IE11 (Grid partiellement supporté)
<code>grid-template-areas</code> pour un layout lisible comme un dessin	Sur-utiliser Grid pour des composants simples

A. Les bases : `display: grid`

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 1fr;  
  grid-template-rows: auto auto;  
  gap: 1rem;  
}
```

- `grid-template-columns` définit les **colonnes**.
- `grid-template-rows` définit les **lignes**.
- `gap` gère l'espace entre lignes et colonnes.

B. Unités `fr` et fonction `repeat()`

1. L'unité `fr` (fraction)

```
.container {  
  grid-template-columns: 1fr 2fr 1fr; /* 25 % / 50 % / 25 % */  
}
```

`1fr` = une **fraction** de l'espace disponible. Très puissant pour des layouts responsifs sans calcul de pourcentages.

2. La fonction `repeat()`

```
.container {  
  grid-template-columns: repeat(4, 1fr);  
  /* équivalent à : 1fr 1fr 1fr 1fr */  
}
```

C. Galerie responsive magique

```
.galerie {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 1rem;  
}
```

Lecture : crée autant de colonnes que possible, chacune d'au moins 200px, qui se partagent équitablement l'espace restant (`1fr`).

→ La galerie devient **automatiquement responsive** : 4 colonnes sur grand écran, 3 sur tablette, 2 sur mobile large, 1 sur petit mobile. **Sans media query**.

♦ Le trio magique à mémoriser

```
display: grid; grid-template-columns: repeat(auto-fit, minmax(XXXpx, 1fr)); gap: Yrem;
```

Ajustez `XXX` selon la largeur minimale souhaitée d'une carte. Galerie de cartes responsive en 3 lignes.

auto-fit VS auto-fill

- `auto-fit` : les cartes existantes s'**étirent** pour remplir l'espace restant. **À préférer** dans 99 % des cas.
- `auto-fill` : crée des pistes invisibles même sans contenu — utile pour un alignement strict sur une grille virtuelle.

D. Placement des éléments

1. `grid-column` et `grid-row`

```
.carte-large {  
  grid-column: 1 / 3; /* de la colonne 1 à 3 (2 colonnes) */  
  grid-row: 1 / 2;  
}  
.carte-haute {  
  grid-column: 3;  
  grid-row: 1 / 3; /* s'étend sur 2 lignes */  
}
```

2. `span` — alternative plus lisible

```
.carte-large {  
  grid-column: span 2; /* s'étend sur 2 colonnes à partir de sa position naturelle */  
}
```

3. `justify-items` et `align-items` en Grid

Centrer le contenu à l'intérieur de chaque cellule :

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  justify-items: center; /* centré horizontalement dans la cellule */  
  align-items: center; /* centré verticalement dans la cellule */  
}
```

E. `grid-template-areas` — layout nommé

Manière **visuelle** et lisible de décrire un layout :

```
.app {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 60px 1fr 40px;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  min-height: 100vh;  
}  
  
.app > header { grid-area: header; }  
.app > aside { grid-area: sidebar; }  
.app > main { grid-area: main; }  
.app > footer { grid-area: footer; }
```

```
<div class="app">  
  <header>...</header>  
  <aside>...</aside>  
  <main>...</main>  
  <footer>...</footer>  
</div>
```

→ Dashboard complet avec header, sidebar, contenu, footer — **en CSS lisible comme un dessin.**

F. Flexbox ou Grid — comment choisir ?

Critère	Flexbox (chap. XI)	Grid (ce chapitre)
Dimension	1D (ligne OU colonne)	2D (lignes ET colonnes)
Cas typique	Navbar, barre d'action, carousel	Dashboard, galerie, page entière
Complexité	Simple	Plus puissant, plus de concepts
Responsive	<code>flex-wrap</code> + <code>flex: 1</code>	<code>auto-fit</code> + <code>minmax</code> (ultra-propre)

◆ Règle rapide

- **Un seul sens** (horizontal OU vertical) → Flexbox.
- **Deux sens** (tableau, dashboard, page) → Grid.
- **Combinable** : un Grid dont chaque cellule est un Flexbox — parfaitement légitime.

Conclusion

1. À retenir

- `display: grid` + `grid-template-columns: repeat(N, 1fr)` + `gap` = grille basique.
- Unité `fr` = fraction de l'espace disponible, parfaite pour le responsive.
- `repeat(auto-fit, minmax(XXXpx, 1fr))` = galerie auto-responsive sans media query.
- `grid-column: span N` pour qu'un élément couvre plusieurs colonnes.
- `grid-template-areas` décrit un layout **visuellement comme un dessin**.

2. Pour aller plus loin

- **MDN — CSS Grid (guide complet)**
- **CSS Tricks — A Complete Guide to Grid**
- **CSS Grid Garden** — jeu d'apprentissage en français (30 min, **à faire**).